

Credentials Stuffing Attack Prevention Using Machine Learning

Md Mahmuduzzaman Kamol, Md Shamim Siddiky, Fahim Anwar, Al-Monaem Khan, and Abdus Salam

Department of Computer Science

American International University-Bangladesh, Dhaka, Bangladesh

{20-42651-1,20-42649-1,20-42305-1,20-42650-1}@student.aiub.edu, abdu.salam@aiub.edu

Abstract—Millions of compromised databases are traded online daily, allowing hackers to exploit users' accounts and posing a significant threat to online security by jeopardizing individuals' sensitive information. There is an urgent need for enhanced cybersecurity measures and increased public awareness to mitigate these risks. This study highlights the significant concern of account cracking stuffing attacks, a prevalent cybersecurity threat that endangers online systems. Our research implemented machine learning methodologies on web access logs to develop a proactive approach for detecting and preventing unauthorized access attempts. We propose a novel solution that utilizes machine learning on web access logs to enhance account security dynamically and adaptively. By effectively using the DBSCAN clustering algorithm to detect discrete user behavior clusters, we categorize user activities into normal and potentially malicious actors, providing critical insights for preventing credential stuffing. The implications of our findings are substantial for cybersecurity, presenting a proactive and effective measure to enhance online account security. Organizations can secure user data and reduce unauthorized access risks by detecting and thwarting account cracking attempts in real-time. This research adds value to cybersecurity by introducing a novel machine learning-based method for scrutinizing web access, providing a tool to stay ahead of cyber threats and fortify user accounts.

Index Terms—Credentials Stuffing, Account Cracking, Threat Detection, Cybersecurity.

I. INTRODUCTION

Keeping user accounts safe is a big deal in today's world, where we do almost everything online. With more people using the internet for all sorts of things, the risk of account cracking, known as credential stuffing (a cyber-attack), is increasing [1]. People and organizations have tons of important stuff online, so keeping their accounts safe from hackers is super important. The old ways of protecting accounts are not enough anymore because hackers are getting smarter, and their methods are changing. This research strengthens the defenses against account cracking by examining how visitors use websites.

Usually, to keep an account safe, people use passwords and change them from time to time [2]. But this is not enough to stop the clever tricks that hackers use today. Account cracking is a standard attack where hackers find weak spots in how security is set up, which can put the private information in user accounts at risk. A big problem is that many people use the same password for different websites. Sure, it is easier to manage, but it also means that if a hacker gets one password, they might be able to access other accounts, too. This study understands that cracking accounts is a complex issue. We are

not just looking at how hackers might use patterns of how people visit websites but also at how people manage their passwords.

This paper aims to find a novel way to stop hackers from cracking accounts. We will do this by looking at how people usually use websites. If we notice something odd happening, we might be able to catch the hackers before they break in. To do this, we use web access log files, which shows how people visit and use websites.

The main issue we are addressing is a common but risky habit: using the same password for multiple websites. While it seems convenient, it creates a significant security vulnerability. Here is the problem: if a hacker manages to break into one account, and the same password is used elsewhere, they suddenly have access to all those accounts. This pattern of reusing passwords is like hitting the jackpot for cybercriminals. They rely on this tendency, making accessing various accounts much more manageable without being detected. The consequence? An increasing number of account cracking cases endangering personal and confidential information. Our goal is to close this security loophole. We aim to develop an effective strategy that uses machine learning techniques to prevent account cracking, focusing mainly on the challenge posed by the widespread habit of reusing passwords across different online platforms.

In summary, we mainly make the following contributions:

- Introduction of a robust web access log file analysis methodology utilizing diverse clustering algorithms.
- Identification of the most suitable algorithm for detecting potential attackers using web server logs.
- Fine-tuning the identified algorithm to achieve heightened precision in detecting anomalous patterns, enhancing overall user account security.
- In-depth exploration of how clustering algorithms surpass current solutions in identifying and preventing account cracking activities.

II. LITERATURE REVIEW

Account cracking, or unauthorized access to user accounts, poses a significant threat in the digital landscape [3]. As technology evolves, malicious actors employ advanced methods to breach security defenses. One of the most common techniques is brute forcing, where attackers systematically attempt various combinations of usernames and passwords to gain access.

Various prevention methodologies, from password complexity requirements to advanced techniques like multi-factor authentication and behavioral analysis, have been proposed to counter brute force attacks, highlighting the persistent threat of account cracking [4].

This literature review traces the evolution of account cracking prevention methodologies, highlighting key developments and challenges. It also sheds light on the steps leading to our solution, which focuses on identifying attackers employing brute force methods to gain unauthorized access.

Understanding attacker behavior patterns, our solution provides a proactive security approach, enhancing user account protection and deterring brute-force attacks. This review explores critical advancements shaping current prevention methodologies and sets the stage for future innovations in combating account cracking.

A. User Password Reuse

An early challenge in account security was users reusing passwords across multiple platforms, providing attackers easy access to various services. Studies found that 43-51% of users reuse passwords across sites [5]—early efforts aimed to educate users on the importance of unique passwords.

Despite extensive advice on password creation, vulnerabilities persist due to password reuse, posing substantial risks. If a hacker gains access to one account, they can access others using the same password, compromising more secure systems [6].

Research on password security began in 1979 [7]. Users often reuse passwords due to cognitive limits [9], which makes secure accounts as vulnerable as the weakest one [9]. This risk is exacerbated by the proliferation of sites and sophisticated attacks [10].

Accounts sharing the same password are vulnerable [11]. Rogue services might gather credentials to test on other websites. While some hope users choose better passwords for valuable accounts [12], there's little empirical evidence and scarce research on high-value passwords [13], prompting our further investigation.

B. Account Brute-force

Brute-force attacks involve systematically checking all possible passwords until the correct one is found using various hashing techniques like MD5, LM hash, or SHA-1 [14]. The time taken depends on password length and strength [15], as the attack tries every combination of letters, numbers, and symbols.

Researchers developed machine learning models to predict passwords, introducing cross-site password-guessing algorithms with a 30% success rate, surpassing the 14% of standard algorithms [16]. While defenses like PwdHash [17] are proposed, password reuse remains a challenge [18]. Existing solutions against brute-force attacks include CAPTCHA implementation, but attackers have found ways to bypass these using AI and machine learning.

C. CAPTCHA Implementation

CAPTCHA emerged as a widely adopted solution to counter automated brute-force attacks, using challenges that require human-like cognitive abilities. CAPTCHA methods include:

- Text-based CAPTCHAs
- Image-based CAPTCHAs
- Audio-based CAPTCHAs
- Motion-based CAPTCHAs
- Hybrid CAPTCHAs

While effective, users need technical expertise to install and maintain these blockers, which few possess [19]. CAPTCHAs aim to be 100%

D. CAPTCHA Solver Challenge

CAPTCHAs can be bypassed using AI or image processing algorithms. For example, the EZ-Gimpy CAPTCHA was solved with an 83%

E. No Pervasive Studies on Account Cracking

With CAPTCHAs being bypassed, prevention techniques like IP blocking have been suggested, which is our primary research focus. However, little research has been done on these methods. There's a notable gap in comprehensive studies analyzing the patterns of account cracking attacks, especially in identifying attackers through web access log analysis. Thus, we delved into analyzing web access logs to blacklist IPs attempting brute-force attacks.

F. Motivation for Our Proposed Method

Recognizing this critical gap, our research sought to address this deficiency by proposing a novel solution. By developing methodologies for detecting malicious activities within web access logs, our approach aims to add a valuable layer of defense against account cracking. Understanding that IP blocking is an effective countermeasure, we propose a solution that analyzes web access logs to identify malicious requests and ban offending IPs from future access.

III. METHODOLOGY

Our suggested method, shown in Figure 1, uses a machine learning model and a shared repository of malicious IP addresses to improve security for user login procedures. The first thing the system does when a user tries to log in to any application (Application 1 or Application 2) is to see if the IP address is known to be malicious. The login request is instantly rejected if the IP address is discovered in the repository, preventing any potential unwanted access. The login request is sent to the server for additional processing if the IP address is not in the repository. The ML model simultaneously examines the IP address and logs into the central repository in case of any malicious occurrence.

The central repository and the ML model must be integrated for this security measure to be effective. The repository functions as a dynamic database that changes with every login attempt and holds the IP addresses of known malicious individuals. By using clustering methods, the machine

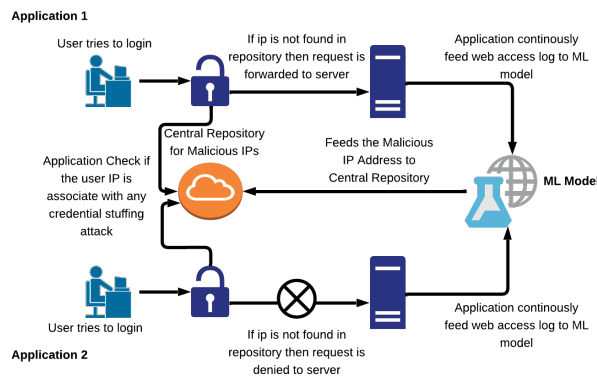


Fig. 1. Our Proposed Method

learning model examines the logged IP addresses and the access patterns that go along with them to spot new dangers and regularly add fresh information to the repository. By continuously learning, the system becomes more capable of anticipatory blocking suspicious IP addresses and adapting to new threatening activities.

The solution guarantees a thorough and dynamic defensive mechanism against malicious login attempts by distributing this centralized repository among several apps, improving overall security. Collective intelligence reduces the risk of security breaches for several applications. By working together, we can ensure that when one program flags an IP address as dangerous, other applications using the same repository will quickly identify and block it. However, it is not possible to implement such a large-scale research project without sufficient funding. Therefore, we decided to start working on the core components of our proposed method and aim to complete the other components progressively. As a result, we concentrated on the first stage, where we attempted to identify attackers through their actions (web access logs), using the shared repository and machine learning models to improve security gradually.

A. Data Collection and Processing

Access to web access logs can pose significant privacy concerns due to the presence of sensitive and personally identifiable information. Moreover, the availability of such logs varies across websites, complicating access for research purposes. To address these challenges, we utilized a synthetic dataset engineered to mimic the statistical characteristics and patterns of web access logs.

Synthetic datasets offer a viable solution for model training, algorithm validation, and research endeavors when genuine data is either sensitive or unavailable. We have used the Python “faker” module to generate the fictitious data across various attributes, including IP addresses, user agents, geographic locations, timestamps, and URLs.

The details of the attributes available in our dataset:

- IP: IP address of the client that made the request;

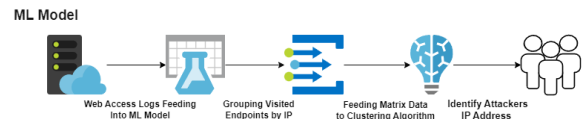


Fig. 2. Applying ML Model to identify attackers

- User Agent: Contains information about the user’s browser, device, and operating system, stored as a character string;
- Time: date and time of the request;
- URL: request type and resource being requested; and
- Byte size: the object’s size returned to the client.

In the following, we present an example of a web access log entry (from our dataset):

```
116.35.41.41 -- [21/May/2022:11:22:41 +0000]
"Get /aboutus.html HTTP/1.1" 200 6430
"http://34.227.9.153/" "Mozilla/5.0
(Macintosh; Intel Mac OS x 10_15_7)
AppleWebKit/605.1.15 (KHTML, like Gecko)
Version/15.4 Safari/605.1.15"
```

In our case, synthetic data is used because real-world web access logs are either confidential or unavailable for specific attacks and benign scenarios. By generating synthetic data, we create a diverse and extensive dataset to simulate both normal and malicious behaviors, which is crucial for training and evaluating machine learning models without needing sensitive or specific real-world data. We have used the below algorithm (see Algorithm 1) to create our web access logs to emulate typical user behavior and malicious activities. It generates log entries with random information, such as User Agent, IP address, and URI, and classifies each entry as either an ‘attack’ or ‘benign.’ The process involves creating benign entries to represent regular user activity and entries based on specific attack patterns to simulate malicious actions. Log generation is executed in parallel using multithreading to enhance performance, where each thread generates and labels logs accordingly. This parallel approach speeds up the creation of a comprehensive dataset for effectively training and testing machine learning models to differentiate between benign and harmful behaviors.

We generated a total of 40,374 web access log entries. Of these, 20,239 entries were created using the Attacker function to simulate malicious behavior patterns, while the User function generated the remaining 20,135 entries to represent typical, benign user activities. Each log entry includes randomized data fields such as User Agent, Country, IP address, Timestamp, Bytes Sent, and URI, along with an appropriate label (‘Attack’ or ‘Benign’) to distinguish between simulated attacks and regular user interactions. These labels were added to evaluate the accuracy of our experimental results. However, during our research, we did not use the label field when feeding data to the ML model; it was used afterward to assess the effectiveness of different clustering algorithms.

Algorithm 1 Simulating Web Access Log Data

Require: N : Number of datapoints, P_u : User probability, P_a : Attack probability, $criticalURIs$: List of critical URIs

```
1: function GENERATE datapoint( $label, ip, uri, userAgent, bytes$ )
2:    $timestamp \leftarrow$  Current Time
3:   Log  $\{label, ip, URI, userAgent, bytes, timestamp\}$ 
4: end function
5: function GENERATE USER
6:    $ip \leftarrow$  Random public IP,  $userAgent \leftarrow$  Random User Agent
7:   for  $i \leftarrow 1$  to Random(5, 20) do
8:      $uri \leftarrow$  Random URI,  $bytes \leftarrow$  Random Bytes Sent
9:     Call GENERATE datapoint('Benign',  $ip, uri, userAgent, bytes$ )
10:  end for
11: end function
12: function GENERATE ATTACK
13:    $criticalURIs \leftarrow$  Assign Critical URIs
14:    $attackType \leftarrow$  Random choice: (Same IP, Multiple IPs)
15:   if  $attackType =$  Same IP then
16:      $ip \leftarrow$  Random public IP
17:     for  $i \leftarrow 1$  to Random(50, 200) do
18:       Call GENERATE datapoint('Attack',  $ip$ , Random choice from  $criticalURIs$ , Random User Agent, Random Bytes Sent)
19:     end for
20:   else
21:     for  $ip \in$  Random selection of IPs do
22:       for  $i \leftarrow 1$  to Random(50, 200) do
23:         Call GENERATE datapoint('Attack',  $ip$ , Random choice from  $criticalURIs$ , Random User Agent, Random Bytes Sent)
24:       end for
25:     end for
26:   end if
27: end function
28: function GENERATE DATA
29:   if Random(0, 1)  $\leq P_u$  then
30:     Call GENERATE USER()
31:   else
32:     Call GENERATE ATTACK()
33:   end if
34: end function
35: for  $i \leftarrow 1$  to  $N$  do
36:   Call GENERATE DATA()
37: end for
```

B. Algorithm for Identifying Clusters for User Behavior

Our algorithm (see Algorithm 2) is designed to identify clusters of user behavior, distinguishing between benign and attack IP addresses using web access log data. The procedure begins with uploading the web access log data and encoding

Algorithm 2 Identifying User Behavior Clusters

```
1: procedure IDENTIFY CLUSTERS(web access log data)
2:   Upload web access log data
3:   Encode categorical data with numerical values
4:   Sort the dataset by IP and URL for sequential analysis
5:   // Aggregate data by IP addresses
6:   for all Unique IP addresses in the log do
7:     Group data by IP
8:   end for
9:   // Create a matrix representing URL visit patterns
10:  Encode URL visit patterns into a matrix
11:  Apply the clustering algorithm
12:  return identified user behavior clusters
13: end procedure
```

categorical data into numerical values. The dataset is sorted by IP and URL to enable sequential analysis. Data is aggregated by unique IP addresses, and then URL visit patterns are encoded into a matrix. A clustering algorithm is applied to group (cluster) user behavior patterns. The algorithm outputs identified user behavior clusters, helping to differentiate between benign and potentially malicious (attack) IP addresses based on their access patterns.

IV. EXPERIMENTAL RESULTS

In cybersecurity, distinguishing between 'benign' and 'attack' activities is vital. In our analysis, we primarily focused on distinguishing these two types of activities through clustering algorithms, as these algorithms group (clusters) similar data based on the specific behavior of the data. Our idea is that since the web access patterns for benign and attackers are different, the clustering algorithms will distinguish the benign and attackers' data into two groups, leading to identifying them using web access log data. After analyzing the publicly available credential stuffing tools aka cracking tools like Sentry MBA [26], SNIPR [26], and OpenBullet [25], we found that most of the configuration files of these tools tend to follow some specific patterns as their primary focus is on account takeover via feeding a large number of account credentials. This might be a key to identifying the attacker's pattern, as they tend to use a single configuration while targeting a specific website. Our research took a unique approach, as we could not obtain a specific dataset for this task. Instead, we created an artificial dataset that combined benign and attack entries.

We have selected popular clustering algorithms for identifying the attackers: K-means, DBSCAN, Gaussian Mixture Model, BIRCH, OPTICS, and Agglomerative Hierarchical Clustering. Our target is to find the algorithm that best suits our dataset and find the attackers with higher accuracy. K-means is a popular method that groups data into clusters based on their center points [27]. It works well with smaller datasets but assumes that clusters are spherical, which is not always the case with web log data that can be noisy

TABLE I
COMPARISON OF DIFFERENT ALGORITHMS FOR IDENTIFYING ‘BENIGN’
AND ‘ATTACKER’

Algorithm	% Accuracy
DBSCAN	90.56%
KMEANS	49.84%
BIRCH	0.24%
OPTICS	49.73%
GAUSSIAN METRICS	50.15%
HIERARCHICAL	49.73%

and irregular. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) stands out because it can find outliers and clusters of any shape, making it very effective for handling the noise and varied patterns in web logs [28]. DBSCAN demonstrated the highest accuracy (90.56%) in identifying attackers within the dataset, outperforming all other clustering algorithms tested. This exceptional performance is attributed to its density-based clustering approach, which effectively identifies clusters of varying shapes and sizes, making it particularly suitable for the noisy and irregular nature of web log data. The Gaussian Mixture Model uses multiple Gaussian distributions to cluster data, which can handle non-circular data but might struggle with noise [29]. BIRCH is suitable for large datasets and focuses on numerical values, creating summaries for efficient grouping; however, it may not handle outliers [30]. In contrast, BIRCH, designed for numeric summarization, failed to handle mixed feature types and lacked robust outlier detection, resulting in negligible accuracy. OPTICS, another density-based method, is good at discovering clusters in data with varying densities, but it can be very complex to use [31]. Although OPTICS shares similarities with DBSCAN, its increased complexity and sensitivity to parameter tuning hindered its performance. Agglomerative Hierarchical Clustering builds clusters in a bottom-up manner and works well for weaker clusters but is less effective with noisy data [32]. Similarly, hierarchical clustering struggled with scalability and noise sensitivity, making it unsuitable for large, noisy datasets.

Table I shows the accuracy of identifying ‘benign’ and ‘attacker’ for all the selected clustering algorithms. We could not compare our results with others as, to the best of our knowledge, no literature works on the concepts with a similar dataset. From the results (see Table I), we observe that DBSCAN best fits our use case and shows an accuracy of 90.56%, which means DBSCAN distinguishes 90.56% attackers correctly. It effectively identifies outliers and handles clusters of any shape, ensuring we can uncover meaningful patterns and anomalies in the data, leading to valuable insights. As mentioned earlier, the dataset contains 20,135 benign and 20,239 attack entries, resulting in 40,374 entries. In each case, there are two clustering groups; if a benign-labeled data point is grouped into cluster 0, it is a successful grouping, but if an attacker-labeled data point is grouped into cluster 0, it is a false positive case. Accordingly, if a benign-labeled data point is grouped into cluster 1, it is a wrong grouping,

but if an attacker-labeled data point is grouped into cluster 1, it is a true positive case. In the case of DBSCAN, there comes a new scenario since it can also identify outliers. If a data point cannot be grouped based on its pattern, DBSCAN declares it as an outlier, and we can see that almost all benign labeled data points are grouped as outliers because there is no such pattern. In the case of attacker-labeled data points, it successfully grouped them into two clusters except for a small portion missing out as outliers. Unlike K-Means and Gaussian Mixture Models, DBSCAN does not assume clusters have spherical or Gaussian distributions. These methods are highly noise-sensitive, whereas DBSCAN adapts to non-linear and complex patterns. Its ability to detect and isolate outliers is invaluable, as attacker behaviors often appear as distinct anomalies from benign patterns.

V. CONCLUSION

This research has embarked on an insightful journey, exploring the critical issue of account cracking in the digital era. Through a detailed analysis of web access log files and optimizing the DBSCAN clustering algorithm, we have developed a methodology capable of detecting anomalous visitation patterns that indicate potential security breaches. Our findings highlight the significant impact of password reuse on account vulnerability and the effectiveness of DBSCAN in enhancing cybersecurity measures. The study underscores the need for continuous innovation in cybersecurity strategies to combat the evolving tactics of cyber attackers.

Looking ahead, we will shift towards creating a more comprehensive and proactive defense mechanism against account cracking. We plan to develop a global platform that functions as a centralized repository for IP addresses flagged by our model as potential security threats. This platform will be a crucial resource in the fight against combo list brute force attacks. Key features of the future work include:

- Building a centralized IP address repository for a dynamic understanding of global threat patterns.
- Developing APIs for wider accessibility and integration with login modules, enhancing widespread adoption.
- Implementing real-time threat analysis for immediate response to new threats.
- Collaborating with cybersecurity communities for shared intelligence and enhanced effectiveness.
- Incorporating adaptive learning mechanisms using machine learning for evolving threat detection.

This initiative aims not just to defend individual accounts but to elevate the security posture of online services globally, creating a more secure digital environment for all users.

REFERENCES

- [1] V. Bulakh, A. J. Kaizer, and M. Gupta, “All your accounts are belong to us,” in *Security and Privacy in Communication Networks (SecureComm 2017)*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 238, G. Pernul, P. Y. A. Ryan, and E. Weippl, Eds. Cham, Switzerland: Springer, 2018, pp. 245–269.

- [2] K. P. L. Vu, R. W. Proctor, A. Bhargav-Spantzel, B. L. Tai, J. Cook, and E. E. Schultz, "Improving password security and memorability to protect personal and organizational information," *Int. J. Hum.-Comput. Stud.*, vol. 65, no. 8, pp. 744–757, Aug. 2007.
- [3] H. Kocabas, S. Nandy, T. Tamanna, and M. N. Al-Ameen, "Understanding user's behavior and protection strategy upon losing or identifying unauthorized access to online account," in *Human-Computer Interaction. Interaction Techniques and Novel Applications (HCII 2021)*, Lecture Notes in Computer Science, vol. 12765, J. Y. C. Chen and G. Fragomeni, Eds. Cham, Switzerland: Springer, 2021, pp. 310–325.
- [4] K. Thomas *et al.*, "Protecting accounts from credential stuffing with password breach alerting," in *28th USENIX Security Symposium (USENIX Security 2019)*, Santa Clara, CA, USA, Aug. 2019, pp. 1556–1571.
- [5] J. Das, M. Bonneau, N. Caesar, N. Borisov, and X. Wang, "The tangled web of password reuse," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS 2014)*, San Diego, CA, USA, Feb. 2014.
- [6] H. Harreld, "Security: An uneasy alliance," *InfoWorld*, vol. 23, no. 13, pp. 42–44, Mar. 2001.
- [7] R. Morris and K. Thompson, "Password security: A case history," *Commun. ACM*, vol. 22, no. 11, pp. 594–597, Nov. 1979.
- [8] S. M. T. Haque, M. Wright, and S. Scielzo, "A study of user password strategy for multiple accounts," in *Proc. 3rd ACM Conf. Data Appl. Secur. Privacy (CODASPY 2013)*, San Antonio, TX, USA, Feb. 2013, pp. 173–176.
- [9] A. Adams and M. A. Sasse, "Users are not the enemy," *Commun. ACM*, vol. 42, no. 12, pp. 40–46, Dec. 1999.
- [10] CERT, "CERT incident note IN-98.03: Password cracking activity," CERT Coordination Center, Carnegie Mellon University, Pittsburgh, PA, USA, 1998. [Online]. Available: https://resources.sei.cmu.edu/asset_files/IncidentNote/1998_2_01.pdf
- [11] D. V. Bailey, M. Dürmuth, and C. Paar, "Statistics on password reuse and adaptive strength for financial accounts," in *Security and Cryptography for Networks (SCN 2014)*, Lecture Notes in Computer Science, vol. 8642, M. Abdalla and R. De Prisco, Eds. Cham, Switzerland: Springer, 2014, pp. 218–235.
- [12] C. Herley, P. C. van Oorschot, and A. S. Patrick, "Passwords: If we're so smart, why are we still using them?" in *Financial Cryptography and Data Security (FC 2009)*, Lecture Notes in Computer Science, vol. 5628, R. Dingleline and P. Golle, Eds. Berlin, Germany: Springer, 2009, pp. 230–237.
- [13] "General problem solver." *InstructionalDesign.org*. [Online]. Available: <http://www.instructionaldesign.org/theories/general-problem-solver.html> [Accessed: Oct. 20, 2023].
- [14] H. Kumar, G. Singh, J. Chhabra, and N. Singh, "Rainbow table to crack password using MD5 hashing algorithm," in *Proc. 2013 IEEE Conf. Inf. Commun. Technol. (ICT 2013)*, Thuckalay, India, Apr. 2013, pp. 433–439.
- [15] P. G. Kelley *et al.*, "Guess again (and again and again): Measuring password strength by simulating password-cracking algorithms," in *2012 IEEE Symposium on Security and Privacy*, San Francisco, CA, USA, May 2012, pp. 523–537.
- [16] J. Das, M. Bonneau, N. Caesar, N. Borisov, and X. Wang, "The tangled web of password reuse," in *Proc. Network and Distributed System Security Symposium (NDSS 2014)*, San Diego, CA, USA, Feb. 2014.
- [17] C. Ross, N. Jackson, D. Miyake, D. Boneh, and J. C. Mitchell, "Stronger password authentication using browser extensions," in *Proc. 14th USENIX Security Symposium*, Baltimore, MD, USA, Aug. 2005.
- [18] J. Das, M. Bonneau, N. Caesar, N. Borisov, and X. Wang, "The tangled web of password reuse," in *Proc. Network and Distributed System Security Symposium (NDSS 2014)*, San Diego, CA, USA, Feb. 2014.
- [19] R. Hofstede, M. Jonker, A. Sperotto, and A. Pras, "Flow-based web application brute-force attack and compromise detection," *J. Netw. Syst. Manag.*, vol. 25, no. 4, pp. 735–758, Oct. 2017.
- [20] G. Mori and J. Malik, "Recognizing objects in adversarial clutter: Breaking a visual CAPTCHA," in *Proc. 2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2003)*, Madison, WI, USA, Jun. 2003, vol. 1, pp. I-134–I-141.
- [21] G. Keizer, "Spammers' bot cracks Microsoft's CAPTCHA: Bot beats Windows Live Mail's registration test 30% to 35% of the time, says Websense," *Computerworld*, Feb. 7, 2008.
- [22] S. Prasad, "Google's CAPTCHA busted in recent spammer tactics," *Websense*, Feb. 22, 2008. [Online]. Available: [ActualURL] [Accessed: Dec. 21, 2008].
- [23] M. Kumar, M. K. Jindal, and M. Kumar, "A systematic survey on CAPTCHA recognition: Types, creation and breaking techniques," *Arch. Comput. Methods Eng.*, vol. 29, no. 2, pp. 1107–1136, Mar. 2022.
- [24] M. Wadhwa, N. Mehta, A. Shrivastava, and R. Dube, "CAPTCHA bypass and prevention mechanisms: A review," *IOSR J. Comput. Eng.*, vol. 22, no. 3, pp. 23–29, May–Jun. 2020.
- [25] C. Dodi, "Cyber security's new challenges under COVID-19 pandemic: Between technique and law," *Studii Juridice Universitare*, no. 1–2, p. 35, 2021.
- [26] G. Giese, "Think like a hacker: Reducing cyber security risk by improving API design and protection," *Cyber Secur.: A Peer-Rev. J.*, vol. 4, no. 1, pp. 48–57, Fall 2020.
- [27] T. S. Madhulatha, "Comparison between k-means and k-medoids clustering algorithms," in *Advances in Computing and Information Technology (ACIT 2011)*, Communications in Computer and Information Science, vol. 198, D. C. Wyld, J. Zizka, and D. Nagamalai, Eds. Berlin, Germany: Springer, 2011, pp. 472–481.
- [28] A. F. Omer, M. M. Hassan, S. U. Rehman, N. Kumar, and M. H. Alazab, "Big data mining using K-means and DBSCAN clustering techniques," in *Big Data Analytics and Computational Intelligence for Cybersecurity*, S. U. Rehman, F. Hussain, M. Khalid, and A. Malik, Eds. Cham, Switzerland: Springer, 2022, pp. 231–246.
- [29] H. Fu, Y. Cheng, and W. Huang, "A novel variational robust filter with Gaussian mixture model for unknown non-Gaussian noises," *Measurement*, vol. 221, p. 113417, Nov. 2023.
- [30] J. Harrington and M. Salibián-Barrera, "Finding approximate solutions to combinatorial problems with very large data sets using BIRCH," *Comput. Stat. Data Anal.*, vol. 54, no. 3, pp. 655–667, Mar. 2010.
- [31] P. Bhattacharjee and P. Mitra, "A survey of density-based clustering algorithms," *Front. Comput. Sci.*, vol. 15, no. 1, pp. 1–27, Feb. 2021.
- [32] M. F. Balcan, Y. Liang, and P. Gupta, "Robust hierarchical clustering," *J. Mach. Learn. Res.*, vol. 15, pp. 3831–3871, 2014.